CASE STUDY · GROUP 16 · COMPILER DESIGN

AVX-512 SIMD Vectorization with Conditional Control Flow

Removing branches with mask registers: an innovative solution built with the C-- (C Minus Minus) programming language, checked against compiler output and real AVX-512 hardware.

```
for (int i = 0; i < N; i++) {
    if (A[i] > 0.0f) {
        B[i] = sqrtf(A[i]);
    } else {
        B[i] = 0.0f;
    }
}
```

THE SOLUTION IN THREE INSTRUCTIONS (GCC 13, AVX-512)

```
vcmpps   $1, (%rdi,%rax), %zmm1, %k1   ; k1 = (A > 0) for 16 lanes: the "if"
vsqrtps  (%rdi,%rax), %zmm0{%k1}{z}    ; sqrt where k1 = 1, else 0.0: both branches
vmovups  %zmm0, (%rcx,%rax)            ; store 16 results, no data-dependent branch
```

18–21×
fewer dynamic instructions per element

32×
fewer branches per element

0
data-dependent branch mispredictions

≈ 19×
measured speedup on random data (3.4× on predictable data)

SUBMITTED BY

Ronit Mexson 24BAI0036

Mit Kumar Raju Panchal 24BAI0279

Garv Gupta 24BDS0169

SUPERVISOR

Prof. Hariharan R L

INSTITUTION

School of Computer Science & Engineering (SCOPE)

DATE

October 2026

Contents

1 Problem statement
2 Approach and methodology
3 Question 1 — Why CMP + JMP breaks SIMD
4 Question 2 — Masked vector operations
5 Question 3 — Instruction count and performance
6 Conclusions and recommendations
7 Reproducibility
A Appendix: C-- SIMD lab source
B Appendix: C-- program output
C Appendix: AVX-512 intrinsics kernels
R References

Abstract

A loop containing an `if` / `else` cannot be executed by SIMD hardware as written: a conditional jump steers one instruction pointer, while a 512-bit vector processes sixteen elements that may each need a different path. This case study shows how AVX-512 replaces the jump with a *per-lane predicate* held in a mask register (`k1` – `k7`), so both branches become a single straight-line instruction stream, and measures what that is worth.

The solution is **innovative in method**: every claim is established three independent ways. (1) A **C-- "SIMD lab"**, written in our own memory-safe language, models `zmm` registers and mask registers as safe values, executes the loop four ways, proves they agree lane by lane, simulates two branch predictors and estimates instruction counts and cycles. (2) **GCC 13's actual output** shows how a production compiler applies masking, and when it refuses to. (3) A **hardware benchmark** on an Intel Xeon (Cascade Lake) times six implementations on five data distributions and directly captures the AVX-512 frequency penalty.

Masking cuts dynamic instructions by 18–21× and branches by 32×, removes every data-dependent misprediction, and makes run time independent of the data: about 19× faster than the branchy scalar loop on random data and 3.4× on predictable data. The results also show that, on this CPU, 256-bit masking beats 512-bit masking because the square-root unit is the bottleneck and 512-bit instructions lower the core clock.

1. Problem statement

A data-analysis kernel contains conditional control flow inside a loop (the kernel on the cover page). The case study asks us to:

1. Explain why traditional branching instructions (CMP + JMP) break SIMD pipelining and prevent vectorization.
2. Demonstrate how the compiler uses **masked vector operations** (AVX-512 mask registers `k1` – `k7`, or SIMD blend instructions) to vectorize the loop without explicit branching.
3. Evaluate the instruction count and performance impact when branch misprediction penalties are eliminated via vector predication.

We were asked to solve the problem in an innovative way. We do this by using the C-- language we built, and by checking the model against the real compiler and real hardware.

2. Approach and methodology

Table 1. Three independent lines of evidence

Part	Artifact	What it establishes
A. C-- SIMD lab	`masked_sqrt.cmm` (Appendix A)	A software model of 16-lane `zmm` registers and `k` masks. Runs the loop as scalar-branchy, AVX-512 masked, AVX-512 max-predicated and AVX2 blend; proves the results identical; simulates a 2-bit and a two-level branch predictor; counts instructions and estimates cycles.
B. Compiler evidence	GCC 13 assembly for the original loop	What a production compiler emits for scalar, AVX2 and AVX-512, and which flags make vectorization legal.
C. Hardware measurement	`bench_avx512.c`, `avx2_blend.c` (Appendix C)	Timings of six implementations on five data distributions, IEEE corner-case checks, and a frequency-license experiment.

2.1 Why C--?

C-- (C Minus Minus) is a statically typed, memory-safe imperative language with no pointers, no null, immutable-by-default bindings and bounds-checked collections. Because C-- has no pointers or inline assembly, a vector register *has* to be modelled as a value, `array<float>(16)`, and a mask register as `array<bool>(16)`. Each AVX-512 instruction becomes a small pure function (`vcmp_gt_zero`, `vsqrt_maskz`, `vload_maskz`, `vstore_mask`), so the lane-level semantics of predication can be read, executed and tested directly.

The language's safety rules also demonstrate one of masking's key benefits. When the tail mask `k2` is deliberately removed from the load, C-- does not silently read past the array as C would. It stops with `runtime error: index out of range (index 100, length 100)` (error R006). The very fault that AVX-512 masked loads exist to prevent becomes visible.

2.2 Experimental environment

Item	Value
CPU	Intel Xeon @ 2.80 GHz, Cascade Lake (Skylake-SP core); AVX-512F/DQ/CD/BW/VL/VNNI; 4 vCPUs (shared VM)
Compiler	GCC 13.3 (Ubuntu 24.04)
C-- toolchain	C-- v0.2.0 reference interpreter on Python 3.13
Problem size	N = 100 in the C-- lab (6 full vectors + 4-lane tail); N = 65 537 on hardware (4 096 vectors + 1-lane tail; fits in L2)
Timing	Time-stamp counter (constant 2.8 GHz). Kernels interleaved per trial; median of 15 trials, each the best of 20 runs
Datasets	Random sign (50% positive), 90% positive, all positive, alternating sign, sorted (negatives then positives)

Hardware performance counters are not exposed in this virtual machine. Instruction and branch counts are therefore taken from the disassembly of the exact binaries, and run times from the time-stamp counter (TSC). Because the VM is shared, scalar timings vary between runs; Table 5 gives the typical value and the observed range over more than ten runs. The vector timings were stable to within about 2%.

3. Question 1 — Why CMP + JMP breaks SIMD

At `-O2`, GCC compiles the loop into the following hot loop. The branch to `.L11` depends on the data in every iteration:

```
.L5:  movss   (%rdi,%rax), %xmm0     ; load A[i]
      comiss  %xmm1, %xmm0           ; CMP  A[i] with 0.0
      jbe     .L11                   ; JMP  if A[i] <= 0     <-- data-dependent branch
      sqrtss  %xmm0, %xmm0           ; then-path
      movss   %xmm0, (%rsi,%rax)
      addq    $4, %rax
      cmpq    %rax, %rcx
      jne     .L5
.L11: pxor    %xmm0, %xmm0           ; else-path: B[i] = 0
      movss   %xmm0, (%rsi,%rax)
      ...
```

ANSWER TO QUESTION 1

1. **One program counter, many lanes.** A SIMD instruction applies one operation to 16 lanes at once, but a jump moves the *single* instruction pointer. When lanes disagree (some `A[i] > 0`, some not), no jump target is right for all of them: the branch is *divergent*. Vectorization requires the compiler to remove the control flow first (**if-conversion**), turning the branch into data: a per-lane predicate.
2. **The branch serializes the pipeline.** The CPU must predict `jbe` before `comiss` completes. On random data it guesses wrong about half the time, and each wrong guess flushes the speculatively executed work and restarts fetch. We measured about **19–23 TSC ticks per misprediction** (≈ 20 core cycles, §5.3), several times the cost of the useful work per element.
3. **Hidden side effects block if-conversion.** If-conversion computes `sqrtf` on lanes the program never asked for, which is only legal when that has no visible effect. `sqrtf` of a negative number may set `errno`, so GCC refuses to vectorize until `-fno-math-errno` is given (Table 2). Floating-point exception flags and out-of-bounds accesses in the final partial vector are the other side effects that predication must suppress.

Table 2. GCC 13 vectorizer decisions for the original loop

Command	Result
`gcc -O2`	Scalar loop with `comiss` + `jbe`
`gcc -O3 -mavx512f -mprefer-vector-width=512`	**Not vectorized**: 0 packed sqrt instructions (`errno` semantics)
`gcc -O3 -mavx2 -fno-math-errno`	Vectorized with 32-byte vectors (compare + `vandps` blend)
`gcc -O3 -mavx512f -mprefer-vector-width=512 -fno-math-errno`	**Vectorized with 64-byte vectors** using mask register `k1`

4. Question 2 — Masked vector operations

4.1 What the compiler emits

With `-O3 -mavx512f -mprefer-vector-width=512 -fno-math-errno`, GCC if-converts the loop and predicates it with mask register `k1`:

```
.L4:  vcmpps   $1, (%rdi,%rax), %zmm1, %k1   ; k1[l] = (0.0 < A[i+l])  for 16 lanes
      vsqrtps  (%rdi,%rax), %zmm0{%k1}{z}    ; sqrt where k1 = 1, 0.0 where k1 = 0
      vmovups  %zmm0, (%rcx,%rax)            ; store 16 results
      addq     $64, %rax
      cmpq     %rax, %rsi
      jne      .L4                            ; only branch left: loop-back, always predictable
```

ANSWER TO QUESTION 2

- `vcmpps … %k1` evaluates the `if` condition for all 16 lanes and stores it as 16 bits in mask register `k1`. The CMP survives as data; the JMP is gone.
- `{%k1}`: the square root only takes effect in lanes whose mask bit is 1. Masked-off lanes raise no floating-point exceptions, because AVX-512 suppresses faults on inactive lanes.
- `{z}` **(zero-masking)** writes 0.0 into inactive lanes. That *is* the `else` branch, executed by the same instruction at no extra cost. Merge-masking (without `{z}`) would keep the destination's previous value instead.
- Both branches collapse into one instruction stream, and the loads fold into the memory operands of `vcmpps` and `vsqrtps`.
- A second mask `k2` handles the final partial vector, so no scalar clean-up loop is needed and memory past the array is never touched (our intrinsics version, Appendix C).

4.2 The masks traced lane by lane in C--

The C-- lab executes these instructions as pure functions. For N = 100 the last vector starts at element 96 and has 4 valid lanes:

Table 3. Last vector of the random dataset, from the C-- lab (Appendix B)

Lane	A[i]	k2 (lane exists)	k1 = k2 AND (A > 0)	B[i] = zmm2 lane
0	−61.1725	1	0	0.0
1	58.6853	1	1	7.6606
2	−5.3441	1	0	0.0
3	−17.7061	1	0	0.0
4–15	—	0	0	not stored

Read as integers (`kmovw`): `k2 = 15` (0b1111) and `k1 = 2` (0b0010). The program checks that the masked result equals the scalar branch for every element of every dataset.

The heart of the C-- model is just two instruction functions:

```
// vcmpps k1{k2}, zmm1, zmm0, _CMP_GT_OQ   (zmm0 = 0.0; result ANDed with k2)
fn vcmp_gt_zero(v: array<float>, k: array<bool>) -> array<bool> {
    mut m = array<bool>(lanes512(), false);
    for (mut l = 0; l < lanes512(); l = l + 1) { m[l] = k[l] && v[l] > 0.0; }
    return m;
}

// vsqrtps zmm2{k1}{z}, zmm1   (zero-masking IS the else-branch: inactive lanes become 0.0)
fn vsqrt_maskz(v: array<float>, k: array<bool>) -> array<float> {
    mut r = array<float>(lanes512(), 0.0);
    for (mut l = 0; l < lanes512(); l = l + 1) { if (k[l]) { r[l] = sqrt_nr(v[l]); } }
    return r;
}
```

4.3 Without mask registers: compare + blend (AVX2)

Without `k` registers the predicate is a vector of all-ones/all-zeros lanes. The square root runs on *every* lane (negative lanes produce NaN and set the invalid flag), and a blend selects the result. Because the `else` value is 0.0, GCC uses a bitwise `vandps` , which is cheaper than `vblendvps` :

```
.L4:  vsqrtps   (%rdi,%rax), %ymm1          ; sqrt of all 8 lanes, including negatives -> NaN
      vcmpltps  (%rdi,%rax), %ymm2, %ymm0   ; ymm0 = all-ones where 0 < A
      vandps    %ymm1, %ymm0, %ymm0         ; blend: keep sqrt where all-ones, else 0.0
      vmovups   %ymm0, (%rcx,%rax)
```

Without masks the remaining `N mod 8` elements need a separate scalar loop, which still contains the branch.

4.4 Innovation: algebraic predication

Because `sqrt(0) = 0` , the whole `if` is equivalent to `B[i] = sqrt(max(A[i], +0.0))` . `vmaxps` returns its *second* operand when either input is NaN or both are zero, so `max(A, +0.0)` maps negatives, `-0.0`

and NaN to `+0.0`: exactly the else-branch, bit for bit. We verified this on 0, −0, ±NaN, ±∞ and denormals. The condition disappears into arithmetic; only the tail mask `k2` remains.

5. Question 3 — Instruction count and performance

5.1 Dynamic instruction count

Table 4. Instructions and branches per element, counted from the real hot loops

Implementation	Instr. / iteration	Elements / iteration	Instr. / element	Branches / element	Data-dependent branches / element
Scalar, `if` + `jbe` (gcc -O2)	8	1	8.000	2.000	1.000
AVX2 compare + blend (gcc -O3)	7	8	0.875	0.125	0
AVX2 blend, hand-written	9	8	1.125	0.125	0 (+ scalar tail)
AVX-512 masked (gcc -O3)	**6**	**16**	**0.375**	**0.0625**	**0**
AVX-512 masked, intrinsics	7	16	0.4375	0.0625	0

- Masking cuts the dynamic instruction count by **about 18–21×** (8.0 → 0.375–0.44 per element).
- Branches drop **32×** (2.0 → 0.0625 per element). The one that remains is the loop-back branch, which mispredicts once, at loop exit.
- **Data-dependent branches drop from one per element to zero**, so speed no longer depends on the data.
- The C-- lab reaches the same result for N = 100: **800 → 46 instructions** and **200 → 7 branches**.

5.2 Measured performance

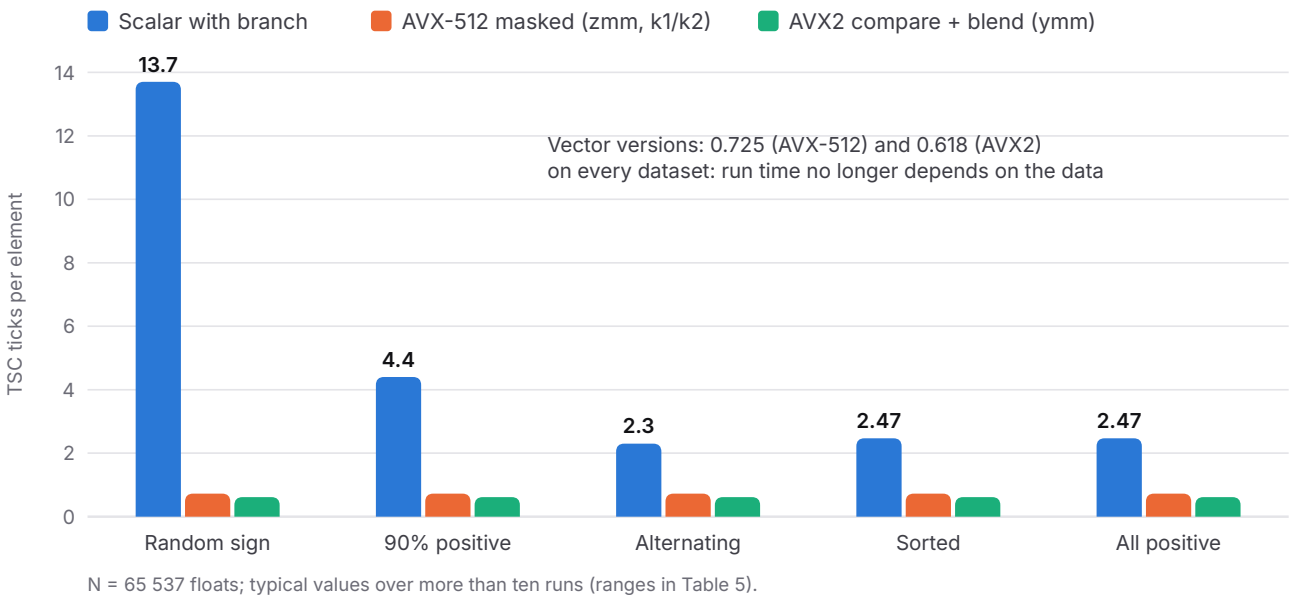


Figure 1. Time per element for the branchy scalar loop and the two predicated vector loops. The scalar cost is set by the branch predictor; the vector cost is constant.

Table 5. Time per element (TSC ticks; lower is better). All results are bit-identical to the scalar loop.

Dataset	Scalar with branch, typical (range)	AVX-512 masked	AVX2 blend	Speedup (AVX-512 masked)
Random sign (50%)	**13.7 (13.7–16.5)**	**0.725**	**0.618**	**≈ 19×**
90% positive	4.4 (4.4–6.6)	0.725	0.618	≈ 6×
Alternating sign	2.3 (2.3–4.0)	0.725	0.618	≈ 3.2×
Sorted (negatives, then positives)	2.47 (2.47–4.7)	0.725	0.618	≈ 3.4×
All positive	2.47	0.725	0.618	≈ 3.4×

The compiler-generated AVX-512 loop and the max-predicated version run at the same 0.725 ticks per element as the masked intrinsics: all three are limited by the same unit (§5.5).

5.3 The cost of branch mispredictions

- The scalar loop's time varies by up to **about 6×** (2.3 → 13.7 ticks) depending only on the *signs* of the data. Every vector version takes the same time on every dataset.
- **Penalty per misprediction.** With random signs about half of the branches mispredict, since no predictor can learn random data. The alternating dataset does the same amount of work (half the elements need a square root) but is perfectly predictable. Hence (13.7 − 2.3) / 0.5 ≈ **23 TSC ticks per misprediction**; the 90% dataset gives ≈ 19. That is about 7–8 ns, or roughly 20 core cycles.
- **83%** of the scalar loop's time on random data is misprediction recovery: (13.7 − 2.3) / 13.7.

5.4 Prediction quality (C-- lab)

Table 6. Data-branch mispredictions for 100 elements: simulated predictors against hardware behaviour

Dataset	2-bit saturating counter	Two-level history predictor	Hardware (ticks / element)
Random sign	52	53	unpredictable (13.7)
90% positive	10	10	about 10% misses (4.4)
Alternating sign	**50**	**4**	**learned (2.3)**
All positive	2	2	no misses (2.47)

A textbook 2-bit counter fails on the alternating pattern; a history-based predictor, the kind real CPUs use, learns it, and the hardware confirms this. Only genuinely unpredictable data exposes the full penalty. **Vector predication removes the penalty in every case**, including the worst one.

5.5 The new bottleneck, and a surprise

Once the branches are gone, the limit is the **square-root unit**, not the number of instructions. Using published reciprocal-throughput figures for Skylake-SP / Cascade Lake, each measurement implies a core clock, and the results are consistent with each other:

Table 7. Square-root throughput and implied core clock

Kernel	Square-root throughput (published)	Measured (ticks / element)	Implied core clock
Scalar `sqrtss`	3 cycles / element	2.471	3.4 GHz
256-bit `vsqrtps ymm`	6 cycles / 8 lanes	0.618	3.4 GHz
512-bit `vsqrtps zmm`	**12 cycles / 16 lanes**	**0.725**	**2.9 GHz**

On this microarchitecture a 512-bit square root has the same per-element throughput as a 256-bit one (0.75 cycles per element), so 512-bit vectors cannot be faster here, and heavy 512-bit floating-point instructions also move the core to a lower frequency “license”. We captured this directly by timing the *same* 256-bit masked kernel before and after a 30 ms burst of 512-bit `vsqrtps`:

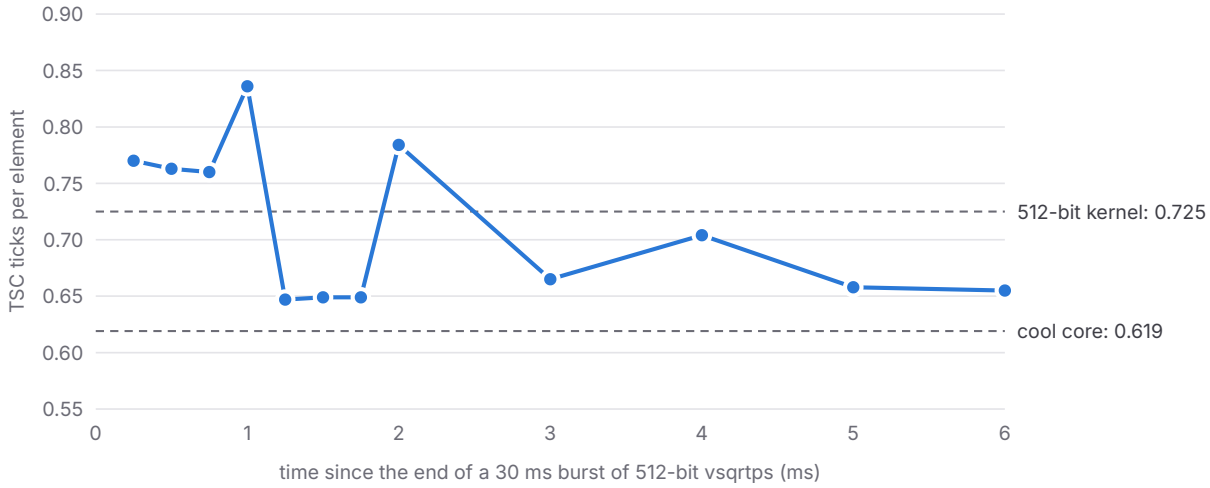


Figure 2. The identical 256-bit masked kernel runs at 0.619 ticks per element on a cool core, slows to about the 512-bit speed for roughly the first millisecond after 512-bit work, then recovers (later fluctuations are VM noise). Data from results/bench_output.txt:

ms after burst	0.25	0.5	0.75	1	1.25	1.5	1.75	2	3	4	5	6
ticks / element	0.770	0.763	0.760	0.836	0.647	0.649	0.649	0.784	0.665	0.704	0.658	0.655

Running the identical masked algorithm at 256-bit width (AVX-512VL) matches AVX2 on a cool core and slows to the 512-bit speed right after 512-bit work. The difference therefore comes from the clock, not from masking. This is consistent with GCC defaulting to `-mprefer-vector-width=256` for `-march=cascadelake`.

5.6 C-- cost model compared with the hardware

Table 8. Prediction of the C-- model and measurement

Case	C-- model (cycles / element, N = 100)	Hardware (TSC ticks / element)
Scalar, random sign	12.8	13.7
Scalar, 90% positive	4.9	4.4
Scalar, alternating sign	3.0	2.3
Scalar, all positive	3.6	2.47
AVX-512 masked, any data	1.04 (0.75 as N → ∞)	0.725

A small cost model written in C-- (issue width, square-root throughput, branch predictor, misprediction penalty) reproduces the ordering and the magnitude of every result. The remaining gap comes from the small N in the model (fixed loop overhead) and from not modelling out-of-order overlap.

ANSWER TO QUESTION 3

Vector predication reduces the dynamic instruction count from **8 to 0.375–0.44 per element** (≈ 18–21×) and the branch count from **2 to 0.0625 per element** (32×), and it eliminates all data-dependent mispredictions. Each misprediction costs about 20 core cycles, and on random data they consume 83% of the scalar loop's time. Removing them gives a measured **≈ 19× speedup on random data and 3.4× on perfectly predictable data**. Above all, run time becomes **independent of the data**: 0.725 ticks per element on every dataset.

6. Conclusions and recommendations

1. **Branches are the obstacle, not the condition.** A CMP + JMP gives all lanes one control flow and makes throughput depend on the predictor. Turning the condition into a per-lane predicate keeps the computation and removes the control flow.
2. **AVX-512 masks make predication complete.** `vcmpps → k1` followed by `vsqrtps{k1}{z}` executes both branches in one instruction: zero-masking supplies the `else`, masked lanes raise no exceptions, and a tail mask `k2` replaces the scalar epilogue without touching memory past the array.
3. **The payoff** is ≈ 18–21× fewer instructions, 32× fewer branches, zero data-dependent mispredictions, and 3.4–19× measured speedups with data-independent run time.
4. **Choose the vector width per CPU.** On Cascade Lake the square-root unit, not the instruction count, is the limit, and 512-bit instructions lower the clock. AVX-512VL masking at 256 bits is the fastest choice here. On CPUs without this frequency penalty (Ice Lake and later, AMD Zen 4) 512-bit masking is expected to win.
5. **Look for algebraic predication.** When the `else` value equals the function at a boundary (here `sqrt(0) = 0`), `sqrt(max(A, 0))` removes the condition altogether.
6. **Compiler flags decide legality.** Without `-fno-math-errno`, `errno` semantics prevent GCC from vectorizing at all.
7. **Safe languages make SIMD semantics testable.** Modelling registers as values in C-- exposed the lane semantics, proved the four strategies equivalent, and turned the out-of-bounds tail access that `k2` prevents into a reported runtime error.

7. Reproducibility

All sources and results are in the C-- repository, folder `case_studies/simd_masked_vectorization/` (github.com/ronitmexson2924/C--). One command regenerates every result:

```
./case_studies/simd_masked_vectorization/run.sh

# or only the C-- SIMD lab (about 0.5 s):
python3 -m cmm run case_studies/simd_masked_vectorization/masked_sqrt.cmm
```

File	Contents
`masked_sqrt.cmm`	C-- SIMD lab (Appendix A)
`bench_avx512.c`, `avx2_blend.c`	Hardware benchmark and AVX2-only kernel
`run.sh`	Builds and runs everything, writes `results/`
`results/cmm_output.txt`	C-- lab output (Appendix B)
`results/unmasked_tail_trap.txt`	C-- run without the tail mask: stopped by error R006
`results/vectorizer_report.txt`, `asm_*.s`	GCC vectorizer decisions and assembly for the original loop
`results/asm_benchmark_kernels.txt`, `bench_output.txt`	Disassembly of every benchmarked kernel; one complete benchmark run

Model notes. C-- floats are 64-bit; the 16-lane width models float32 in a 512-bit register. Square roots in the C-- lab use Newton–Raphson iteration (C-- has no built-in square root), so the program checks $B^2 \approx A$ rather than comparing with a library result. The C99 backend of C-- does not yet support arrays, so the lab runs on the C-- reference interpreter.

Appendix A. C-- SIMD lab source (`masked_sqrt.cmm`)

```
module main;

// ============================================================================
// Case Study (Group 16): AVX-512 SIMD vectorization with conditional control flow
//
//     for (int i = 0; i < N; i++) {
//         if (A[i] > 0.0f) B[i] = sqrtf(A[i]); else B[i] = 0.0f;
//     }
//
// This C-- program is a "SIMD laboratory". C-- has no pointers and no inline
// assembly, so registers are modelled as safe values:
//   * a 512-bit zmm register  = array<float>(16)  (16 lanes of float32)
//   * an AVX-512 mask k1..k7   = array<bool>(16)   (one predicate bit per lane)
// Every vector instruction is a small pure function, so the program can
//   1. run the loop four ways (scalar branchy, AVX-512 masked, AVX-512
//      max-predicated, AVX2 compare+blend) and prove they agree lane by lane,
//   2. simulate the branch predictor that the scalar version depends on,
//   3. count dynamic instructions and estimate cycles for each strategy.
//
// Note: C-- floats are 64-bit; the lane count of 16 models float32 in a zmm.
// Run with:  python3 -m cmm run case_studies/simd_masked_vectorization/masked_sqrt.cmm
// ============================================================================

// ---------------------------------------------------------------- machine model
// Approximate Skylake-SP / Cascade Lake values (Agner Fog, uops.info); the
// mispredict penalty is calibrated from bench_avx512.c (~19-23 TSC ticks measured).
fn lanes512() -> int { return 16; }
fn lanes256() -> int { return 8; }
fn issue_width() -> int { return 4; }
fn mispredict_penalty() -> int { return 20; }
fn sqrtss_cost() -> int { return 3; }      // reciprocal throughput, scalar
fn sqrtps_ymm_cost() -> int { return 6; }  // 8 lanes
fn sqrtps_zmm_cost() -> int { return 12; } // 16 lanes

// Dynamic instructions per loop iteration, counted from the real hot loops that
// gcc 13 emits (see results/asm_*.s): loads fold into memory operands.
fn scalar_iter_then() -> int { return 8; }  // movss, comiss, jbe, sqrtss, movss, add, cmp, jne
fn scalar_iter_else() -> int { return 8; }  // movss, comiss, jbe, pxor, movss, add, cmp, je
fn avx512_masked_iter() -> int { return 6; } // vcmpps [A]->k1, vsqrtps [A]{k1}{z}, vmovups, add, cmp, jne
fn avx512_max_iter() -> int { return 6; }    // vmaxps [A], vsqrtps, vmovups, add, cmp, jne
fn avx512_tail_setup() -> int { return 4; }  // mov, shl/bzhi, dec, kmovw -> k2 (once)
fn avx2_blend_iter() -> int { return 7; }    // vsqrtps [A], vcmpltps [A], vandps, vmovups, add, cmp, jne

// ---------------------------------------------------------------- utilities
fn lcg(state: int) -> int {
    // 31-bit linear congruential generator (fits in checked 64-bit arithmetic)
    return (1103515245 * state + 12345) % 2147483648;
}

fn unit(state: int) -> float {
    return (1.0 * state) / 2147483648.0;  // [0, 1)
}

// Newton-Raphson square root: the model of the hardware sqrt unit.
// Starting at or above sqrt(x), the iteration decreases monotonically,
// so it stops exactly when no further progress is possible.
fn sqrt_nr(x: float) -> float {
    if (x <= 0.0) {
        return 0.0;
    }
    mut g = x;
    if (g < 1.0) {
        g = 1.0;
    }
```

```
    for (mut k = 0; k < 64; k = k + 1) {
        let next = 0.5 * (g + x / g);
        if (next >= g) {
            return g;
        }
        g = next;
    }
    return g;
}

fn pow2(e: int) -> int {
    mut p = 1;
    for (mut i = 0; i < e; i = i + 1) {
        p = p * 2;
    }
    return p;
}

// kmovw eax, k  -> the mask as an integer (bit i = lane i)
fn mask_bits(k: array<bool>) -> int {
    mut bits = 0;
    for (mut l = 0; l < k.len(); l = l + 1) {
        if (k[l]) {
            bits = bits + pow2(l);
        }
    }
    return bits;
}

fn popcnt(k: array<bool>) -> int {
    mut c = 0;
    for (mut l = 0; l < k.len(); l = l + 1) {
        if (k[l]) {
            c = c + 1;
        }
    }
    return c;
}

// ---------------------------------------------------------------- datasets
// pattern 1: random sign (50/50)   pattern 2: 90% positive
// pattern 3: all positive          pattern 4: strictly alternating sign
fn make_input(n: int, pattern: int, seed: int) -> array<float> {
    mut a = array<float>(n, 0.0);
    mut s = seed;
    for (mut i = 0; i < n; i = i + 1) {
        s = lcg(s);
        let magnitude = 0.01 + unit(s) * 99.99;
        s = lcg(s);
        let r = unit(s);
        mut positive = true;
        if (pattern == 1) {
            positive = r < 0.5;
        } else if (pattern == 2) {
            positive = r < 0.9;
        } else if (pattern == 4) {
            positive = i % 2 == 0;
        }
        if (positive) {
            a[i] = magnitude;
        } else {
            a[i] = 0.0 - magnitude;
        }
    }
    a[0] = 0.0;  // boundary case: 0.0 > 0.0 is false, so B[0] must be 0.0
    return a;
}

// ---------------------------------------------------------------- kernel 1: scalar with a branch
fn scalar_branchy(a: array<float>, n: int) -> array<float> {
    mut b = array<float>(n, -1.0);
```

```
    for (mut i = 0; i < n; i = i + 1) {
        if (a[i] > 0.0) {
            b[i] = sqrt_nr(a[i]);
        } else {
            b[i] = 0.0;
        }
    }
    return b;
}

// ------------------------------------------------------------------ AVX-512 instruction models
// Tail mask k2: lanes [0, valid) are active. Computed once for the last vector.
fn make_tail_mask(valid: int) -> array<bool> {
    mut k = array<bool>(lanes512(), false);
    for (mut l = 0; l < lanes512(); l = l + 1) {
        k[l] = l < valid;
    }
    return k;
}

// vmovups zmm1{k2}{z}, [A + base]  (masked load: no out-of-bounds access, inactive lanes = 0)
fn vload_maskz(a: array<float>, base: int, k: array<bool>) -> array<float> {
    mut v = array<float>(lanes512(), 0.0);
    for (mut l = 0; l < lanes512(); l = l + 1) {
        if (k[l]) {
            v[l] = a[base + l];
        }
    }
    return v;
}

// vcmpps k1{k2}, zmm1, zmm0, _CMP_GT_OQ   (zmm0 = 0.0; result ANDed with k2)
fn vcmp_gt_zero(v: array<float>, k: array<bool>) -> array<bool> {
    mut m = array<bool>(lanes512(), false);
    for (mut l = 0; l < lanes512(); l = l + 1) {
        m[l] = k[l] && v[l] > 0.0;
    }
    return m;
}

// vsqrtps zmm2{k1}{z}, zmm1   (zero-masking IS the else-branch: inactive lanes become 0.0)
fn vsqrt_maskz(v: array<float>, k: array<bool>) -> array<float> {
    mut r = array<float>(lanes512(), 0.0);
    for (mut l = 0; l < lanes512(); l = l + 1) {
        if (k[l]) {
            r[l] = sqrt_nr(v[l]);
        }
    }
    return r;
}

// vmaxps zmm1, zmm1, zmm0   (max(x, 0): negative and zero lanes collapse to 0.0)
fn vmax_zero(v: array<float>) -> array<float> {
    mut r = array<float>(lanes512(), 0.0);
    for (mut l = 0; l < lanes512(); l = l + 1) {
        if (v[l] > 0.0) {
            r[l] = v[l];
        }
    }
    return r;
}

// vsqrtps zmm2, zmm1   (unmasked)
fn vsqrt(v: array<float>) -> array<float> {
    mut r = array<float>(v.len(), 0.0);
    for (mut l = 0; l < v.len(); l = l + 1) {
        r[l] = sqrt_nr(v[l]);
    }
    return r;
}
```

```
// vmovups [B + base]{k2}, zmm2   (masked store; value semantics return the new B)
fn vstore_mask(b: array<float>, base: int, v: array<float>, k: array<bool>) -> array<float> {
    mut out = b;
    for (mut l = 0; l < lanes512(); l = l + 1) {
        if (k[l]) {
            out[base + l] = v[l];
        }
    }
    return out;
}

// ------------------------------------------------------------------ kernel 2: AVX-512 masked (k1/k2)
fn avx512_masked(a: array<float>, n: int) -> array<float> {
    mut b = array<float>(n, -1.0);
    for (mut base = 0; base < n; base = base + lanes512()) {
        mut valid = n - base;
        if (valid > lanes512()) {
            valid = lanes512();
        }
        let k2 = make_tail_mask(valid);
        let zmm1 = vload_maskz(a, base, k2);
        let k1 = vcmp_gt_zero(zmm1, k2);
        let zmm2 = vsqrt_maskz(zmm1, k1);
        b = vstore_mask(b, base, zmm2, k2);
    }
    return b;
}

// ------------------------------------------------------------------ kernel 3: AVX-512 algebraic
predication
// B = sqrt(max(A, 0)) is the same function as the branch: sqrt(0) = 0.
// The condition disappears into arithmetic; only the tail needs a mask.
fn avx512_max(a: array<float>, n: int) -> array<float> {
    mut b = array<float>(n, -1.0);
    for (mut base = 0; base < n; base = base + lanes512()) {
        mut valid = n - base;
        if (valid > lanes512()) {
            valid = lanes512();
        }
        let k2 = make_tail_mask(valid);
        let zmm1 = vload_maskz(a, base, k2);
        let zmm2 = vsqrt(vmax_zero(zmm1));
        b = vstore_mask(b, base, zmm2, k2);
    }
    return b;
}

// ------------------------------------------------------------------ kernel 4: AVX2 compare + blend (no
mask registers)
// vcmpltps ymm0 = (0 < A) as all-ones/all-zeros lanes; vsqrtps computes EVERY lane
// (negative lanes produce NaN and raise the invalid flag); vandps (or vblendvps)
// keeps sqrt where the lane is all-ones and yields 0.0 elsewhere.
// The n % 8 remainder runs through a scalar epilogue, as compilers generate.
fn avx2_blend(a: array<float>, n: int) -> array<float> {
    mut b = array<float>(n, -1.0);
    let full = n - n % lanes256();
    for (mut base = 0; base < full; base = base + lanes256()) {
        for (mut l = 0; l < lanes256(); l = l + 1) {
            let x = a[base + l];
            let select = x > 0.0;      // vcmpltps lane mask
            let s = sqrt_nr(x);        // vsqrtps (NaN lanes modelled as discarded)
            if (select) {              // vandps/vblendvps: per-lane select, no branch in hardware
                b[base + l] = s;
            } else {
                b[base + l] = 0.0;
            }
        }
    }
    for (mut i = full; i < n; i = i + 1) {
        if (a[i] > 0.0) {
            b[i] = sqrt_nr(a[i]);
```

```
        } else {
            b[i] = 0.0;
        }
    }
    return b;
}

// ------------------------------------------------------------------ verification
fn same(x: array<float>, y: array<float>) -> bool {
    if (x.len() != y.len()) {
        return false;
    }
    for (mut i = 0; i < x.len(); i = i + 1) {
        if (x[i] != y[i]) {
            return false;
        }
    }
    return true;
}

fn semantics_hold(a: array<float>, b: array<float>) -> bool {
    for (mut i = 0; i < a.len(); i = i + 1) {
        if (a[i] > 0.0) {
            let err = b[i] * b[i] - a[i];
            if (err > 0.000000001 * a[i] || err < 0.0 - 0.000000001 * a[i]) {
                return false;
            }
        } else if (b[i] != 0.0) {
            return false;
        }
    }
    return true;
}

// ------------------------------------------------------------------ branch predictors
fn bump(counter: int, taken: bool) -> int {
    if (taken && counter < 3) {
        return counter + 1;
    }
    if (!taken && counter > 0) {
        return counter - 1;
    }
    return counter;
}

// Textbook 2-bit saturating counter for the `if (A[i] > 0)` branch.
fn mispredicts_2bit(a: array<float>, n: int) -> int {
    mut counter = 2;
    mut misses = 0;
    for (mut i = 0; i < n; i = i + 1) {
        let actual = a[i] > 0.0;
        if ((counter >= 2) != actual) {
            misses = misses + 1;
        }
        counter = bump(counter, actual);
    }
    return misses;
}

// Two-level local-history predictor (4 history bits -> 16 two-bit counters),
// the family modern CPUs build on: it learns repeating patterns such as
// alternating signs, but nothing can learn truly random data.
fn mispredicts_two_level(a: array<float>, n: int) -> int {
    mut pht = array<int>(16, 2);
    mut history = 0;
    mut misses = 0;
    for (mut i = 0; i < n; i = i + 1) {
        let actual = a[i] > 0.0;
        if ((pht[history] >= 2) != actual) {
            misses = misses + 1;
        }
```

```
        pht[history] = bump(pht[history], actual);
        mut bit = 0;
        if (actual) {
            bit = 1;
        }
        history = (history * 2 + bit) % 16;
    }
    return misses;
}

// ------------------------------------------------------------------ cost models
// Returns [instructions, data-branch mispredicts, loop mispredicts, sqrt ops, branches]
fn scalar_cost(a: array<float>, n: int) -> array<int> {
    mut instructions = 0;
    mut sqrt_ops = 0;
    for (mut i = 0; i < n; i = i + 1) {
        if (a[i] > 0.0) {
            instructions = instructions + scalar_iter_then();
            sqrt_ops = sqrt_ops + 1;
        } else {
            instructions = instructions + scalar_iter_else();
        }
    }
    return [instructions, mispredicts_two_level(a, n), 1, sqrt_ops, 2 * n];
}

fn vectors(n: int, width: int) -> int {
    return (n + width - 1) / width;
}

// Returns [instructions, data-branch mispredicts, loop mispredicts, sqrt ops, branches]
fn avx512_cost(n: int, iter: int) -> array<int> {
    let v = vectors(n, lanes512());
    mut setup = 0;
    if (n % lanes512() != 0) {
        setup = avx512_tail_setup();
    }
    return [v * iter + setup, 0, 1, v, v];
}

fn avx2_cost(a: array<float>, n: int) -> array<int> {
    let full = n - n % lanes256();
    let v = full / lanes256();
    mut instructions = v * avx2_blend_iter();
    mut mispredicts = 0;
    mut branches = v;
    mut counter = 2;
    mut scalar_sqrt = 0;
    for (mut i = full; i < n; i = i + 1) {
        let actual = a[i] > 0.0;
        if ((counter >= 2) != actual) {
            mispredicts = mispredicts + 1;
        }
        if (actual) {
            if (counter < 3) { counter = counter + 1; }
            instructions = instructions + scalar_iter_then();
            scalar_sqrt = scalar_sqrt + 1;
        } else {
            if (counter > 0) { counter = counter - 1; }
            instructions = instructions + scalar_iter_else();
        }
        branches = branches + 2;
    }
    return [instructions, mispredicts, 2, v, branches, scalar_sqrt];
}

// cycles = max(front-end issue bound, sqrt-unit bound) + mispredicts * penalty
fn cycles(instructions: int, sqrt_cycles: int, mispredicts: int) -> int {
    mut issue = (instructions + issue_width() - 1) / issue_width();
    if (sqrt_cycles > issue) {
        issue = sqrt_cycles;
```

```
    }
    return issue + mispredicts * mispredict_penalty();
}

fn per_element(total: int, n: int) -> float {
    return (1.0 * total) / n;
}

// ------------------------------------------------------------- report
fn report(name: string, n: int, pattern: int, seed: int) -> int {
    let a = make_input(n, pattern, seed);
    let ref = scalar_branchy(a, n);
    let m512 = avx512_masked(a, n);
    let x512 = avx512_max(a, n);
    let b256 = avx2_blend(a, n);

    print("------------------------------------------------------------");
    print(name);
    print("  elements / AVX-512 vectors (16 lanes) / tail lanes:");
    print(n);
    print(vectors(n, lanes512()));
    print(n % lanes512());

    print("  verified: masked == scalar, max == scalar, blend == scalar, B = sqrt(A) or 0:");
    print(same(m512, ref));
    print(same(x512, ref));
    print(same(b256, ref));
    print(semantics_hold(a, ref));

    let sc = scalar_cost(a, n);
    print("  data-branch mispredicts: 2-bit counter / two-level history predictor (used below):");
    print(mispredicts_2bit(a, n));
    print(sc[1]);
    let mk = avx512_cost(n, avx512_masked_iter());
    let mx = avx512_cost(n, avx512_max_iter());
    let bl = avx2_cost(a, n);

    let sc_cycles = cycles(sc[0], sc[3] * sqrtss_cost(), sc[1] + sc[2]);
    let mk_cycles = cycles(mk[0], mk[3] * sqrtps_zmm_cost(), mk[1] + mk[2]);
    let mx_cycles = cycles(mx[0], mx[3] * sqrtps_zmm_cost(), mx[1] + mx[2]);
    let bl_cycles = cycles(bl[0], bl[3] * sqrtps_ymm_cost() + bl[5] * sqrtss_cost(), bl[1] + bl[2]);

    print("  [scalar branchy] instructions / branches / mispredicts / est. cycles / cycles per element:");
    print(sc[0]);
    print(sc[4]);
    print(sc[1] + sc[2]);
    print(sc_cycles);
    print(per_element(sc_cycles, n));
    print("  [AVX-512 masked k1/k2] instructions / branches / mispredicts / est. cycles / cycles per
element:");
    print(mk[0]);
    print(mk[4]);
    print(mk[1] + mk[2]);
    print(mk_cycles);
    print(per_element(mk_cycles, n));
    print("  [AVX-512 max-predicated] instructions / branches / mispredicts / est. cycles / cycles per
element:");
    print(mx[0]);
    print(mx[4]);
    print(mx[1] + mx[2]);
    print(mx_cycles);
    print(per_element(mx_cycles, n));
    print("  [AVX2 blend + scalar tail] instructions / branches / mispredicts / est. cycles / cycles per
element:");
    print(bl[0]);
    print(bl[4]);
    print(bl[1] + bl[2]);
    print(bl_cycles);
    print(per_element(bl_cycles, n));
    print("  speedup of AVX-512 masked over scalar (estimated):");
    print((1.0 * sc_cycles) / mk_cycles);
```

```
    print("  share of scalar cycles lost to mispredicts:");
    print((1.0 * (sc[1] + sc[2]) * mispredict_penalty()) / sc_cycles);
    return sc_cycles;
}

fn trace_tail(n: int, seed: int) -> int {
    // Show the k2 (tail) and k1 (condition) masks for the last, partial vector.
    let a = make_input(n, 1, seed);
    let base = (n / lanes512()) * lanes512();
    let k2 = make_tail_mask(n - base);
    let zmm1 = vload_maskz(a, base, k2);
    let k1 = vcmp_gt_zero(zmm1, k2);
    let zmm2 = vsqrt_maskz(zmm1, k1);
    print("------------------------------------------------------------");
    print("Lane trace: last vector of the random dataset (elements from index):");
    print(base);
    print("  k2 = tail mask as integer (bit i = lane i), active lanes:");
    print(mask_bits(k2));
    print(popcnt(k2));
    print("  k1 = (A > 0) AND k2 as integer, active lanes:");
    print(mask_bits(k1));
    print(popcnt(k1));
    print("  per valid lane: A[i], then B[i] = zmm2 lane:");
    for (mut l = 0; l < n - base; l = l + 1) {
        print(zmm1[l]);
        print(zmm2[l]);
    }
    return mask_bits(k1);
}

fn main() {
    print("C-- SIMD lab: masked vectorization of `if (A[i] > 0) B[i] = sqrt(A[i]); else B[i] = 0;`");
    print("Model: 16-lane zmm registers, k1/k2 mask registers, two-level branch predictor,");
    print("4-wide issue, 20-cycle mispredict penalty, sqrtss 3 / vsqrtps zmm 12 cycles throughput.");
    let n = 100;
    let r1 = report("Dataset 1: random sign (50% positive) - worst case for the predictor", n, 1, 2024);
    let r2 = report("Dataset 2: 90% positive", n, 2, 7);
    let r3 = report("Dataset 3: all positive (except A[0] = 0.0)", n, 3, 11);
    let r4 = report("Dataset 4: alternating sign", n, 4, 5);
    let k = trace_tail(n, 2024);
    print("------------------------------------------------------------");
    print("Scalar cycles across the four datasets (data-dependent), then AVX-512 masked (data-
independent):");
    print(r1);
    print(r2);
    print(r3);
    print(r4);
    print(cycles(avx512_cost(n, avx512_masked_iter())[0], vectors(n, lanes512()) * sqrtps_zmm_cost(), 1));
}
```

Appendix B. C-- program output (dataset 1 and lane trace)

Each `print` writes one value per line (C-- has no string concatenation); each label line is followed by its values in order. Full output: `results/cmm_output.txt`.

```
C-- SIMD lab: masked vectorization of `if (A[i] >
0) B[i] = sqrt(A[i]); else B[i] = 0;`
Model: 16-lane zmm registers, k1/k2 mask
registers, two-level branch predictor,
4-wide issue, 20-cycle mispredict penalty, sqrtss
3 / vsqrtps zmm 12 cycles throughput.
----------------------------------------
Dataset 1: random sign (50% positive) - worst
case for the predictor
  elements / AVX-512 vectors (16 lanes) / tail
lanes:
100
7
4
  verified: masked == scalar, max == scalar,
blend == scalar, B = sqrt(A) or 0:
true
true
true
true
  data-branch mispredicts: 2-bit counter / two-
level history predictor (used below):
52
53
  [scalar branchy] instructions / branches /
mispredicts / est. cycles / cycles per element:
800
200
54
1280
12.8
  [AVX-512 masked k1/k2] instructions / branches
/ mispredicts / est. cycles / cycles per element:
46
7
1
104
1.04
  [AVX-512 max-predicated] instructions /
branches / mispredicts / est. cycles / cycles per
element:
46
7
1
104
1.04
  [AVX2 blend + scalar tail] instructions /
branches / mispredicts / est. cycles / cycles per
element:
116
20
5
175
1.75
  speedup of AVX-512 masked over scalar
(estimated):
12.307692307692308
  share of scalar cycles lost to mispredicts:
0.84375
----------------------------------------
Lane trace: last vector of the random dataset
(elements from index):
96
  k2 = tail mask as integer (bit i = lane i),
active lanes:
15
4
  k1 = (A > 0) AND k2 as integer, active lanes:
2
1
  per valid lane: A[i], then B[i] = zmm2 lane:
-61.17254489399026
0.0
58.68526411209721
7.6606307907441415
-5.34411190548446
0.0
-17.70606234482024
0.0
----------------------------------------
Scalar cycles across the four datasets (data-
dependent), then AVX-512 masked (data-
independent):
1280
493
357
300
104
```

Appendix C. AVX-512 intrinsics kernels (from `bench_avx512.c`)

```
__attribute__((noinline))
void avx512_masked(const float *restrict A, float *restrict B, int n) {
    const __m512 zero = _mm512_setzero_ps();
    int i = 0;
    for (; i + 16 <= n; i += 16) {
        __m512 a = _mm512_loadu_ps(A + i);
        __mmask16 k1 = _mm512_cmp_ps_mask(a, zero, _CMP_GT_OQ); /* k1 = A > 0          */
        __m512 r = _mm512_maskz_sqrt_ps(k1, a);                 /* sqrt where k1, else 0 */
        _mm512_storeu_ps(B + i, r);
    }
    if (i < n) {                                                /* tail: k2 = low (n-i) bits */
        __mmask16 k2 = (__mmask16)((1u << (n - i)) - 1u);
        __m512 a = _mm512_maskz_loadu_ps(k2, A + i);            /* no out-of-bounds read  */
        __mmask16 k1 = _mm512_mask_cmp_ps_mask(k2, a, zero, _CMP_GT_OQ);
        _mm512_mask_storeu_ps(B + i, k2, _mm512_maskz_sqrt_ps(k1, a));
    }
}

__attribute__((noinline))
void avx512_max(const float *restrict A, float *restrict B, int n) {
    const __m512 zero = _mm512_setzero_ps();
    int i = 0;
    for (; i + 16 <= n; i += 16) {
        /* MAXPS returns the 2nd operand for NaN and for +-0, so max(A, +0) maps
           NaN, -0.0 and negatives to +0.0: exactly the else-branch. */
        _mm512_storeu_ps(B + i, _mm512_sqrt_ps(_mm512_max_ps(_mm512_loadu_ps(A + i), zero)));
    }
    if (i < n) {
        __mmask16 k2 = (__mmask16)((1u << (n - i)) - 1u);
        __m512 a = _mm512_maskz_loadu_ps(k2, A + i);
        _mm512_mask_storeu_ps(B + i, k2, _mm512_sqrt_ps(_mm512_max_ps(a, zero)));
    }
}

__attribute__((noinline))
void avx512vl_ymm(const float *restrict A, float *restrict B, int n) {
    const __m256 zero = _mm256_setzero_ps();
    int i = 0;
    for (; i + 8 <= n; i += 8) {
        __m256 a = _mm256_loadu_ps(A + i);
        __mmask8 k1 = _mm256_cmp_ps_mask(a, zero, _CMP_GT_OQ);
        _mm256_storeu_ps(B + i, _mm256_maskz_sqrt_ps(k1, a));
    }
    if (i < n) {
        __mmask8 k2 = (__mmask8)((1u << (n - i)) - 1u);
        __m256 a = _mm256_maskz_loadu_ps(k2, A + i);
        __mmask8 k1 = _mm256_mask_cmp_ps_mask(k2, a, zero, _CMP_GT_OQ);
        _mm256_mask_storeu_ps(B + i, k2, _mm256_maskz_sqrt_ps(k1, a));
    }
}

/* avx2_blend.c (compiled with -march=haswell) */
void avx2_blend(const float *restrict A, float *restrict B, int n) {
    const __m256 zero = _mm256_setzero_ps();
    int i = 0;
    for (; i + 8 <= n; i += 8) {
        __m256 a = _mm256_loadu_ps(A + i);
        __m256 m = _mm256_cmp_ps(a, zero, _CMP_GT_OQ);          /* all-ones lanes where A > 0 */
        __m256 s = _mm256_sqrt_ps(a);                           /* every lane (NaN if A < 0)  */
        _mm256_storeu_ps(B + i, _mm256_blendv_ps(zero, s, m));  /* per-lane select            */
    }
    for (; i < n; i++) {                                        /* scalar epilogue            */
        B[i] = (A[i] > 0.0f) ? sqrtf(A[i]) : 0.0f;
```

```
    }
}
```

References

1. Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Vol. 2 (VCMPPS, VSQRTPS, VMAXPS, VBLENDVPS, opmask registers) and Vol. 1, Ch. 15 (AVX-512 programming).
2. Intel Corporation. *Intel® Intrinsics Guide*: `_mm512_cmp_ps_mask`, `_mm512_maskz_sqrt_ps`, `_mm512_mask_storeu_ps`, `_mm512_maskz_loadu_ps`.
3. Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*: AVX-512 frequency behaviour and branch misprediction.
4. A. Fog. *Instruction tables: lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs.* Technical University of Denmark.
5. A. Abel and J. Reineke. *uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures.* ASPLOS 2019.
6. J. R. Allen, K. Kennedy, C. Porterfield and J. Warren. *Conversion of Control Dependence to Data Dependence.* POPL 1983 (if-conversion).
7. T.-Y. Yeh and Y. N. Patt. *Two-Level Adaptive Training Branch Prediction.* MICRO 1991.
8. GCC Team. *Using the GNU Compiler Collection*: `-fno-math-errno`, `-mprefer-vector-width`, `-fopt-info-vec`.
9. C-- (C Minus Minus) project. *Language Guide and Specification*, v0.2.0. github.com/ronitmexson2924/C--.